TuringLab.

Lesson 12.

Normalization and Initialization

(정규화와 초기화)

개요

학습 과정에서 활성값은 신경망을 따라 전파된다. 그러나 네트워크의 깊이가 깊어질수록, 활성값의 분포는 **층(layer) 간** 또는 **배치(batch) 간**에서 크게 달라질 수 있다. 이러한 현상은 **내부 공변량 변화(internal covariate shift)**라고 불리며, 학습을 불안정하게 만들고 기울기 크기가 크게 흔들리게 한다. 이를 완화하기 위해 **배치 정규화(Batch Normalization)**와 **레이어 정규화(Layer Normalization)** 같은 정규화 기법이 사용된다.

정규화(Normalize)를 하는 이유

학습 과정에서 활성값은 신경망을 따라 전파된다. 그러나 네트워크의 깊이가 깊어질수록, 활성값의 분포는 **층(layer) 간** 또는 **배치(batch) 간**에서 크게 달라질 수 있다. 이러한 현상은 **내부 공변량 변화(internal covariate shift)**라고 불리며, 학습을 불안정하게 만들고 기울기 크기가 크게 흔들리게 한다. 이를 완화하기 위해 **배치 정규화(Batch Normalization)**와 **레이어 정규화(Layer Normalization)** 같은 정규화 기법이 사용된다.

🔎Inernal Covariates

공변량(covariate)은 모델의 **입력으로 사용되는 변수**를 뜻한다.

1. 내부 공변량 (Internal Covariants)이란?

신경망에서 어떤 층을 기준으로 보면, 그 **이전 층의 출력(=현재 층의 입력으로 들어오는 활성값)**들이 바로 **내부 공변량**이다.

- 데이터 $x \rightarrow$ 첫 번째 층의 입력
- 첫 번째 층의 출력 $h_1 \rightarrow$ 두 번째 층의 **내부 공변량**

- 두 번째 층의 출력 $h_2 \rightarrow$ 세 번째 층의 **내부 공변량**

즉, **네트워크 내부에서 각 층이 입력으로 받는 값들**을 내부 공변량이라고 부른다.

2. 내부 공변량 변화 (Internal Covariate Shift)란?

학습이 진행되면서, 앞쪽 층의 파라미터(가중치)가 업데이트되면 그 결과로 **뒤쪽 층이 받는 입력 분포가 계속 변하는 현상**을 말한다.

조금 더 구체적으로 말하면:

- 한 층이 어떤 입력 분포(평균, 분산 등)에 맞춰 학습 중인데
- 이전 층의 가중치가 바뀌면서
- **같은 층의 입력 분포가 매 iteration / batch마다 달라지는 것**

이게 바로 내부 공변량 변화다.

3. 왜 문제가 되는가?

내부 공변량 변화가 크면:

- 각 층이 **계속 바뀌는 입력 분포**에 적응해야 해서 학습이 느려짐
- 활성값이 너무 커지거나 작아져서 $\rightarrow$ **기울기 폭주 / 기울기 소실** 발생 가능
- 학습률을 크게 쓰기 어려워짐
- 초기화에 민감해짐

결과적으로 **학습이 불안정**해진다.

4. 정규화는 무엇을 하는가?

Batch Normalization, Layer Normalization 같은 기법은:

- 각 층으로 들어오는 내부 공변량의
    - 평균을 0 근처로
    - 분산을 1 근처로
- 유지하도록 만들어

입력 분포의 변화 폭을 줄여준다.

> **Note**

활성값(Activation)이란?

각 뉴런이 입력을 받아 가중합을 계산한 뒤, **활성화 함수(activation function)를 통과한 출력값**

수식으로 쓰면:

$$z = Wx + b \quad \text{(선형 변환)}$$

$$a = \phi(z) \quad \text{(활성값)}$$

여기서

- x: 입력
- W, b: 가중치, 편향
- ϕ: `ReLU`, `Sigmoid`, `Tanh` 같은 활성화 함수
- a: **활성값**

활성값의 역할

활성값은 단순한 숫자가 아니라:

- **비선형성(non-linearity)**을 모델에 도입
- 층을 여러 개 쌓아도 의미 있는 표현 학습 가능
- 다음 층의 입력으로 전달되는 **신호(signal)**

📒 비선형성(non-linearity)이란?

모델의 출력이 입력의 **선형 결합으로만 표현되지 않는 성질**을 말한다.

1. 선형과 비선형의 정확한 차이

선형(linear)의 의미

함수 f가 다음을 만족하면 선형이다.

$$f(ax + by) = af(x) + bf(y)$$

신경망에서 선형 층만 쌓으면:

$$y = W_3(W_2(W_1x)) = Wx$$

$\rightarrow$ **아무리 깊어도 결국 하나의 선형 변환**

비선형(non-linear)의 의미

위 성질을 **만족하지 않는 함수**가 비선형 함수다.

예:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid
- Tanh

이 함수들을 통과하면:

$$f(W_2(f(W_1x))) \neq Wx$$

→ **표현력이 급격히 증가**

2. 왜 비선형성이 반드시 필요한가?

핵심 이유

> **현실 세계의 대부분의 문제는 선형으로 표현할 수 없다.**

예를 들어:

- XOR 문제
- 이미지 분류
- 음성 인식
- 자연어 처리

이 문제들은 **선형 결정 경계**로는 절대 분리되지 않는다.

직관적 예: XOR 문제

XOR는 단층 퍼셉트론(선형 모델)로는 절대 풀 수 없다.

비선형 활성화가 들어가면:

- 중간 표현 공간을 비틀고
- 입력을 다른 공간으로 매핑해서
- 선형 분리가 가능해진다

3. 비선형성 = 공간을 "구부리는 것"

직관적으로 말하면:

- 선형 변환:
  → 공간을 **회전, 이동, 스케일링**만 함
- 비선형 변환:
  → 공간을 **접고, 자르고, 구부림**

그래서 복잡한 패턴을 표현할 수 있다.

4. 활성화 함수와 비선형성의 관계

활성화 함수의 역할

- 비선형성을 **의도적으로 주입**하는 장치
- 신경망이 단순한 선형 모델로 붕괴되는 것을 방지

활성화 함수가 없다면:

- 깊은 신경망 = 얕은 선형 모델

Batch Normalization (배치 정규화)

특정 뉴런(또는 특징 채널)에서 나온 활성값들로 이루어진 미니배치 $\{x_1, x_2, \ldots, x_m\}$가 주어졌을 때, **배치 정규화(Batch Normalization)**는 이 값들을 **배치 차원(batch dimension)을 기준으로 정규화**한다.

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{12.1}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{12.2}$$

$$\hat{x} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \tag{12.3}$$

$$y_i = \gamma\widehat{x_i} + \beta \tag{12.4}$$

여기서:

- μ_B 그리고 σ_B^2는 미니배치에서 계산된 평균과 분산이며, $\widehat{x_i}$는 정규화된 활성값이다,
  - m은 표본의 개수. $\{x_1, x_2, \ldots, x_m\}$에서 x는 어떤 집합에서 뽑은 m개의 값.
  - 즉, **평균을 계산할 때 포함되는 데이터 포인트의 수**

    - 그러므로, μ_B는 표본의 평균값이다.
- γ와 β는 다시 스케일링하고 이동시키기 위해 학습되는 파라미터들이며, ε은 수치적 안정성을 위해 더해지는 작은 상수이다.
- (12.3)에서
    - $x_i - \mu_B$는 배치안의 i번째 값(x_i)에서 배치 평균을 빼서, 데이터를 평균 0을 중심으로 이동 (분산을 줄임)
    - $\sqrt{\sigma_B^2 + \varepsilon}$
        - 표준편차로 나누어 스케일을 1로 맞춤
        - $\sqrt{\sigma_B^2}$는 분산의 제곱근이므로 **표준편차**
        - ε은 0으로 나누는 것을 방지하기 위한 작은 값
    - 즉, 데이터의 스케일을 표준편차 1로 맞춤
    - 결과 $\hat{x}$은 정규화된 값
        - 즉, **평균 0, 분산 1을 갖는 형태로 변환된 입력**이다.
- (12.4)에서
    - γ (scale parameter, 학습되는 파라미터)
        - 모델이 학습을 통해 직접 조정하는 **scale(크기)**
        - $\widehat{x_i}$를 얼마나 확대하거나 축소할지 결정
    - β (shift parameter, 학습되는 파마미터)
        - 모델이 학습을 통해 조정하는 **이동(shift)**
        - 출력의 평균을 원하는 값으로 이동
    - **결과**: y_i
        - 정규화된 값 $\widehat{x_i}$를 다시
            - 확대/축소(γ)
            - 이동(β)

          해서 모델이 필요로 하는 분포로 되돌린 값

이런 과정을 추가하는 이유

정규화만 하면 모든 값이 평균 0, 분산 1이 되어 표현력이 제한된다. 그래서 모델이 필요하면 다시 스케일을 조정할 수 있도록 γ와 β를 학습 가능한 파라미터로 둔다.

즉,

- BN은 학습 안정성을 위해 정규화
- γ, β는 표현력 회복을 위해 재조정

이 두 가지가 합쳐져서 BatchNorm이 강력해진다.

미니배치(mini-batch)란?

전체 학습 데이터 중에서 **한 번의 파라미터 업데이트에 사용되는 작은 데이터 묶음**을 말한다.

1. 왜 미니배치를 쓰는가?

신경망 학습은 보통 **경사 하강법(Gradient Descent)**을 사용한다. 이때 데이터 사용 방식은 세 가지가 있다.

① 배치 학습 (Batch)

- 전체 데이터셋을 한 번에 사용
- 장점: 기울기가 정확함
- 단점: 메모리 부담 큼, 느림

② 확률적 경사 하강법 (SGD)

- 데이터 1개씩 사용
- 장점: 빠른 업데이트
- 단점: 기울기 노이즈 큼, 불안정

③ 미니배치 (Mini-batch) ← 실제로 가장 많이 사용

- 전체 데이터 중 **일부(예: 32, 64, 128개)**만 사용
- 속도, 안정성, 메모리 사용의 균형이 좋음

2. 미니배치의 정확한 의미

- 데이터셋 크기: N
- 미니배치 크기: m

한 번의 학습 스텝에서:

$$\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$$

을 사용해

- 손실 계산
- 기울기 계산

- 가중치 업데이트

를 수행한다.

3. Batch Normalization과 미니배치의 관계

BN에서 미니배치는 특히 중요하다.

- BN은 **미니배치 내부의 평균과 분산**을 사용해 정규화
- 그래서:
    - 미니배치가 너무 작으면 → 통계량이 불안정
    - 미니배치가 1이면 → BN이 의미 없음

이 때문에:

- 작은 배치 → LayerNorm, GroupNorm이 선호됨

4. 한 줄 요약

- **미니배치**: 한 번의 학습에 사용하는 데이터 묶음
- **목적**: 계산 효율 + 학습 안정성
- **BN과의 관계**: BN은 미니배치 통계에 의존함

배치놈은 일반적으로 선형 변환(linear transformation) 이후, 비선형 활성함수(non-linearity) 이전에 삽입된다:

$$\text{Linear} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}$$

이는 기울기 흐름을 개선하고, 더 빠른 학습을 가능하게 하며, 더 높은 학습률을 사용할 수 있게 해준다. 그러나, 안정적인 통계치를 얻기 위해서는 충분히 큰 배치 크기에 의존한다.

Layer Normalization (레이어 정규화)

매우 작은 배치를 사용하는 작업이나 **시퀀스 기반 구조**(예: NLP 모델이나 Transformer)에서는, **배치 통계량이 신뢰하기 어렵기 때문에** Batch Normalization이 잘 동작하지 않을 수 있다. 이런 경우에는 **Layer Normalization**이 더 효과적인데, 이는 **배치 전체가 아니라 각 개별 샘플의 특징(feature) 차원에 대해 정규화**를 수행하기 때문이다.

입력 벡터 $x \in \mathbb{R}^d$ (e.g., 샘플 하나에 대한 한 계층의 활성값들)가 주어지면, 계층놈(LayerNorm)은 다음과 같이 계산된다:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \tag{12.5}$$

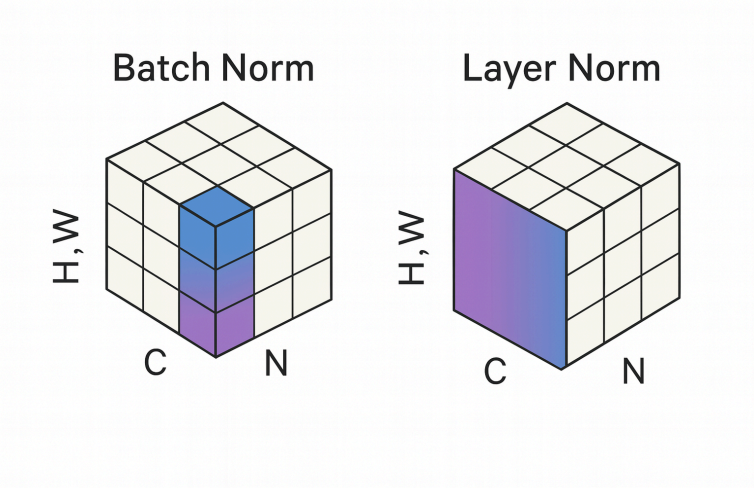
$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 \tag{12.6}$$

$$\hat{x} = \frac{x_i = \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \tag{12.7}$$

$$y_i = \gamma\hat{x}_i + \beta \tag{12.8}$$

여기서, 정규화는 훈련 예제 하나에 대한 입력 벡터의 모든 d개 특성(feature)들에 대해 수행되며, 동일한 학습 가능한 스케일과 이동 파라미터 (γ, β)가 적용된다.

이 방법은 배치 크기와 무관하며 순환 신경망(recurrent networks) 또는 오토그레시브 디코더 같이 샘플 단위로 일관성 있는 동작이 요구되는 모델에서 잘 동작한다. 특히 이 접근법은 입력 패턴들이 매우 다양하고 순차적인 변환기(Transformers) 그리고 순환 신경망에서 매우 효과적이다.



[그림 12.1] 배치 놈(왼쪽)과 레이어 놈(오른쪽) 비교. 배치 놈은 배치 축(N)을 기준으로 수행하는 반면, 레이어 정규화는 각 샘플 내부의 모든 특성(C, H, W)을 기준으로 정규화를 수행한다.

Weight Initialization (가중치 초기화)

훈련이 시작되기 전에 가중치는 일반적으로 무작위 값으로 초기화 된다. 그러나, 부적절한 초기화는 신호 흐름을 왜곡할 수 있고, 기울기 소실이나 폭발, 그리고 학습 지연을 유발할 수 있다. 초기화 목표는 모든 층에서 활성값과 기울기의 분산을 적절하게 통제되도록 관리한다.

Xavier Intialization (자비어 초기화)

`tanh` 또는 `sigmoid`와 같은 대칭적 활성함수들을 위해서 설계된 **Xavier 초기화**는, 다음 분산을 갖는 분포에서 가중치를 샘플링한다:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \tag{12.9}$$

여기서 n_{in}과 n_{out}은 각각 그 계층의 입력과 출력 유닛들의 개수이다.

이것은 순전파와 역전파 과정에서 신호가 증폭되거나 약해지는 것을 방지하도록 신호 흐름을 균형있게 유지해준다.

수식의 의미

- W_{ij}
  - 신경망의 가중치 행렬 W의 한 요소(가중치 하나)를 의미
  - 즉, **i번째 출력 뉴런과 j번째 입력 뉴런을 연결하는 가중치**이다.
- $\sim$
  - 같다(=)는 의미가 아님 (**확률적 관계**를 나타내는 기호)
  - **"~로부터 샘플링된다"**라는 의미
  - 즉, 가중치 W_{ij}는 어떤 확률 분포에서 뽑힌다는 의미

> Wij는 평균이 0이고 분산이 $\frac{2}{n_{\text{in}} + n_{\text{out}}}$인 **정규분포(Normal distribution)**에서 **무작위로 뽑힌 값이다**

> **Important**
>
> ### $\mathcal{N}(0, \sigma^2)$의 의미
>
> 이 식은 정규분포를 나타내는 기호이다. σ^2는 **분산(variance)**을 의미한다.
>
> - σ는 **표준편차(standard deviation)**
> - σ^2는 **표준편차의 제곱 = 분산**
>
> 즉,
>
> $$\mathcal{N}(0, \sigma^2)$$
>
> 은 **평균이 0이고, 분산이 σ^2인 정규분포**를 의미한다.
>
> $$\mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

에서 분산(σ^2)은 $\left(\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$이다.

왜 분산에 제곱을 쓰는가?

정규분포는 보통 **평균(mean)**과 **분산(variance)**으로 정의된다.

- 평균: 분포의 중심
- 분산: 값들이 평균 주변에서 얼마나 퍼져 있는가?

표준편차는 σ는 분산의 제곱근이지만,

정규분포의 수학적 정의에서는 **분산** σ^2가 더 자연스럽게 사용되기 때문에 표기에서도 σ^2를 사용한다.

예시로 이해하기

$$\mathcal{N}(0, 4)$$

이면,

- 평균 = 0
- 분산 = 4
- 표준편차 = $\sqrt{4}$ = 2

즉, 값들이 평균 0을 중심으로 ± 2 정도 범위에서 많이 분포한다는 뜻이다.

Tip

입력 유닛 (Input Units, n_{in})

- 한 층(Layer)이 **받아들이는 값의 개수**
- 즉, **해당 층의 입력 벡터 차원 수**

예시:

- 입력 벡터가 $[x_1, x_2, x_3]$이면, 입력 유닛 수는 3
- 이전 층의 뉴런이 128개라면 다음 층의 입력 유닛 수는 128

출력 유닛 (Output Units, n_{out})

- 한 층이 **내보내는 값의 개수**
- 즉, **해당 층의 출력 벡터 차원 수**

예시:

- Dense layer가 64개의 뉴런을 가진다면 출력 유닛 수는 64
  - Dense layer는 딥러닝에서 가장 기본적이고 널리 사용되는 층(layer) 중 하나
  - 한국어로는 **완전연결층(fully connected layer)** 또는 **전결합층**이라고 함
- 분류 문제에서 클래스가 10개라면 마지막 층의 출력 유닛 수는 10

정리

- **입력 유닛**: 그 층이 받는 값의 수
- **출력 유닛**: 그 층이 만들어내는 값의 수

이 개념이 Xavier 초기화나 He 초기화에서 중요한 이유는, 가중치 분산을 입력·출력 유닛 수에 맞춰 조절해야 신호가 폭발하거나 소실되지 않기 때문이다.

He Initialization

`ReLU`처럼 음수 입력을 0으로 만드는 비대칭 활성함수의 경우, He 초기화는 분산을 다음과 같이 더 크게 설정한다.

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \tag{12.10}$$

이는 ReLU가 만들어내는 희소성(sparity)에도 불구하고 기울기 흐름이 유지되도록 보장한다.

He/Xavier 초기화에서 분산이 중요한 이유

신경망에서 각 층은 다음 층으로 **신호(activation)**를 전달하고, 역전파에서는 **기울기(gradient)**가 반대 방향으로 전달된다. 이때 **가중치 초기화의 분산이 적절하지 않으면** 다음과 같은 문제가 생긴다.

1) 분산이 너무 크면: 신호·기울기 폭발(Explosion)

- 가중치가 너무 큰 값으로 초기화되면

$$y = Wx + b$$

  에서 출력 y가 지나치게 커짐
- 층을 거칠수록 값이 기하급수적으로 커져서 **forward activations 폭발** $\rightarrow$ **gradient 폭발**

2) 분산이 너무 작으면 $\rightarrow$ 신호·기울기 소실(Vanishing)

- 가중치가 너무 작으면 출력이 거의 0에 가까워짐
- 층을 거칠수록 신호가 점점 작아져 **forward activations 소실** $\rightarrow$ **gradient 소실**
- 깊은 네트워크에서 학습이 거의 진행되지 않음

3) 층마다 분산이 유지되어야 안정적인 학습이 가능

이상적인 초기화는 다음을 만족해야 한다.

- **순전파 activations의 분산이 층을 지나도 유지**
- **역전파 gradients의 분산도 층을 지나도 유지**

즉,

$$Var(x^{(l)}) \approx Var(x^{(l+1)})$$
$$Var(\nabla x^{(l)}) \approx Var(\nabla x^{(l+1)})$$

이 조건을 만족시키기 위해 Xavier와 He 초기화가 등장하였음.

- $x^{(l+1)}$: $(l+1)$번째 층의 활성값(activation)
- $\nabla x^{(l+1)}$: 그 활성값에 대한 **손실함수의 기울기**, 즉 역전파로 전달되는 gradient
- $Var(\nabla x^{(l+1)})$
  - 역전파(backpropagation) 과정에서 $(l+1)$번째 층의 기울(gradient)가 가지는 분산을 의미
  - 기울기의 크기가 지나치게 커지거나 작아지지 않는지를 보는 지표

Xavier 초기화가 분산을 조절하는 이유

Xavier는 **대칭 활성함수(`tanh`, `sigmoid`)**를 가정하고, 입력/출력 유닛 수를 고려해 분산을 다음처럼 설정한다.

$$Var(W) = \frac{1}{n_{\text{in}}} \text{ or } \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

신호가 너무 커지거나 작아지지 않도록 균형 유지

He 초기화가 분산을 더 크게 잡는 이유

ReLU는 음수 입력을 0으로 만들어 **출력의 절반이 사라지는 희소성(sparsity)**이 생김. 그래서 Xavier보다 더 큰 분산이 필요하다.

$$Var(W) = \frac{2}{n_{\text{in}}}$$

- ReLU로 인해 줄어드는 신호를 보상
- forward/backward 모두에서 분산 유지

Summary (요약)

심층 신경망은 **활성값과 기울기의 스케일 및 분포**에 매우 민감하다. 정규화, 특히 **Batch Normalization**과 **Layer Normalization**은 특징 값들을 표준화함으로써 불안정을 완화하고, 층 사이에서 정보 흐름이 원활해지도록 돕는다. 한편, **적절한 초기화**는 학습 초기에 기울기가 너무 크거나 너무 작아지는 것을 방지한다.

Xavier 초기화는 대칭적인 활성화 함수를 사용하는 네트워크에 적합하며, **He 초기화**는 ReLU 기반 모델에 맞춰 설계되었다. 이러한 초기화 기법들은 신호 왜곡을 최소화하고 학습 효율을 향상시킨다.

정규화와 초기화는 함께 작동하여, 현대 딥러닝에서 **안정적이고 빠르며 견고한 학습 동역학**의 핵심 기반을 이룬다.

ReLU 다시 보기

ReLU(Rectified Linear Unit, 교정된 선형 단위)는 딥러닝에서 가장 널리 쓰이는 **활성화 함수**다. ReLU는 다음과 같이 정의된다.

$$\text{ReLU}(x) = \max(0, x)$$

즉,

- 입력이 **양수면 그대로 출력**
- 입력이 **0이하이면 0출력**

아주 단순한 형태지만, 이 단순함이 신경망 학습을 크게 안정시키는 역할을 한다.

왜 ReLU가 중요한가?

1) 기울기 소실 문제 (Vanishing Gradient)를 완화

`sigmoid`나 `tanh`는 입력이 크면 기울기가 거의 0이 되어 깊은 네트워크에서 학습이 잘 안된다.

ReLU는 양수 영역에서 기울기가 1이라서, 깊은 층에서도 기울기가 잘 전달된다.

2) 계산이 매우 빠름

- max 연산 하나면 끝
- 미분도 간단: GPU에서 특히 효율적

3) 희소성(Sparsity)을 유도

입력의 절반 정도는 0이 되기 때문에 뉴런이 선택적으로 활성화된다. 이것은 일종의 정규화 효과를 주고, 모델이 더 일반화되도록 돕는다.

ReLU의 단점

1) Dying ReLU 문제

입력이 계속 음수면 뉴런이 영원히 0만 출력하게 되고, 기울기도 0이 되어 학습이 멈춘다.

이를 해결하기 위해 여러가지 변형이 존재한다.

- Leaky ReLU
- Parameteric ReLU
- ELU
- GELU 등

출력 분포가 한쪽으로 치우침

ReLU는 음수 영역을 모두 0으로 만들기 때문에 출력 평균이 양수 쪽으로 치우칠 수 있다. 이 때문에 BatchNorm과 같은 정규화 기법과 함께 쓰면 훨씬 안정적이다.

ReLU가 He 초기화와 자주 함께 사용되는 이유

ReLU는 음수 입력을 절반 정도 죽이기 때문에, 가중치 초기화 시 분산을 더 크게 잡아야 한다. 그래서 He 초기화가 등장했다.

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

이 분산이 ReLU의 특성과 잘 맞아 떨어져서 학습이 안정된다.

Lesson 13

Data Augmentation (데이터 증강)

개요

모델이 안정적으로 학습되고, 매끄럽게 수렴하며, 낮은 학습 손실을 보이더라도 실제 배포 단계에서 크게 실패할 수 있다. 왜 이런 일이 발생할까?
현실 세계의 데이터는 노이즈가 많고 일관성이 부족하며, 학습에 사용한 잘 정제된 샘플과는 닮은 구석이 없다. 보지 못한 입력에서도 좋은 성능을 내는 일반화 능력의 핵심은 항상 네트워크 구조 속에만 숨어 있는 것이 아니다. 종종 그 해답은 데이터 자체에 있다.
데이터 증강(data augmentation)은 레이블을 보존하는 변환을 적용해 데이터셋을 확장하고 다양화하는 과정이다. 이는 실제 환경에서 발생하는 다양한 변화를 시뮬레이션함으로써 모델이 불필요한 변화에 영향을 받지 않도록, 즉 불변성을 갖도록 만들어 모델의 강건성을 높여준다.

언제 증강(Augmentation)이 도움을 주나?

데이터 증강은 학습 데이터셋이 작거나, 불균형하거나, 다양성이 부족할 때 특히 효과적이다. 이러한 상황에서는 모델이 과적합되기 쉬워 학습 데이터에서는 높은 정확도를 보이지만 새로운 입력이나 보지 못한 예제에서는 일반화에 실패한다.
증강은 데이터에 통제된 변형을 적용하여 분포 변화에 대한 모델의 강건성을 높여준다. 이는 테스트 도메인에 학습 데이터에 충분히 포함되지 않은 패턴이나 왜곡이 존재할 때 특히 중요하다.
좋은 증강 전략은 모델이 더 어렵고 다양한 입력을 접하게 되므로 학습 손실이 약간 증가하는 경향이 있지만, 검증 손실은 눈에 띄게 감소하고 OOD(out-of-distribution) 데이터에서 더 높은 정확도를 보이게 한다.
결과적으로 데이터 증강은 일반화와 신뢰성을 향상시키는 간단하면서도 강력한 방법이며, 특히 시각적 작업이나 레이블이 제한된 상황에서 더욱 유용하다.

Augmentation vs. Generation

(증강 vs. 일반화)

증강(augmentation)을 생성(generation)과 구분하는 것은 매우 중요하다. 증강은 새로운 샘플을 무(無)에서 만들어내는 것이 아니다. 대신, 의미적 레이블을 유지한 채 동일한 샘플의 다른 관점을 만들어낸다.
입력 이미지 x와 그에 대응하는 레이블 y가 있다고 하자. 증강은 다음 조건을 만족하는 변환된 버전 $x' = T(x)$를 구성한다.

$$f(x') \approx f(x), \quad \text{and} \quad \text{label}(x') = y \tag{13.1}$$

여기서 T는 허용 가능한 변형(admissible perturbations)의 분포로부터 선택된 변환 함수이며, f는 모델을 의미한다.

Preserving Label Consistency (레이블 일관성 유지)

변환된 샘플이 원래의 레이블을 유지하는 것은 매우 중요한 요구사항이다. 이를 위반하면 학습 과정에 노이즈가 섞여 성능이 저하될 수 있다.

예를 들어, 숫자 "6"을 180°로 회전시키면 시각적으로 "9"와 구분하기 어려운 형태가 된다. 그런데도 이를 "6"으로 라벨링하면 모델을 잘못된 방향으로 학습시키게 된다. 이는 증강이 아니라 의미적 손성(semantic corruption)이다.

Transformations (변환)

시각적 도메인에서는 다음과 같은 변환들이 보편적으로 적용된다.

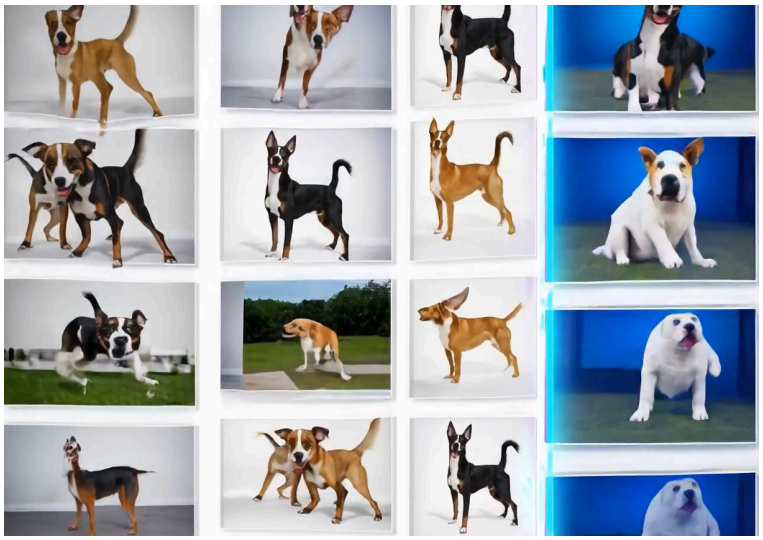
- 수평 뒤집기(Horizontal Flip): $x' = \text{Flip}_H(x)$ — 객체의 대칭성이 허용될 때 적합하다.
- 회전(Rotation): $x' = \text{Rotate}_\theta(x), \theta \in [-15°, 15°]$
- 크롭 및 리사이즈(Crop and Resize): 임의의 부분 영역을 잘라낸 뒤 다시 스케일링한다.
- 평행이동 및 줌(Translation and Zoom): 클래스 정체성을 유지한 채 작은 이동이나 확대/축소를 적용한다.
- 밝기 및 대비(Brightness and Contrast): 조명 수준을 조정한다.
- 색상 변화(Color Jitter): 컬러 이미지의 색조/채도를 변형한다.
- 가우시안 노이즈(Additive Noise): 센서 변동성을 시뮬레이션하기 위해 가우시안 노이즈를 추가한다.

Practical Caution (실용적 주의사항)

과도한 데이터 증강은 오히려 역효과를 낼 수 있다. 변형된 샘플이 더 이상 유효한 입력처럼 보이지 않을 정도로 왜곡되거나 객체 구조가 손실되면, 학습 성능이 저하될 수 있다. 따라서 시각적 검토는 필수적이다.

Implementation (구현)

이미지 증강을 위한 강력한 오픈소스 라이브러리로 Albumentations가 있으며, 이는 PyTorch와 TensorFlow 모두와 매끄럽게 통합된다. 이 변환 파이프라인은 이미지를 무작위로 뒤집고, 회전시키고, 밝기를 조정하고, 정규화하면서도 클래스 의미를 그대로 유지한다.



[그림 13.1] 증강 예제들

Offline vs. Online Augmentation

데이터 증강을 적용하는 전략에는 두 가지가 있다.

- **오프라인 증강(Offline augmentation)**: 새로운 이미지를 미리 생성해 저장하는 방식이다. 디스크 사용량이 증가하지만, 변형의 다양성이 고정된다.
- **온라인 증강(Online augmentation)**: 학습 중에 실시간으로 무작위 변환을 적용한다. 에포크마다 확률적 변동성과 사실상 무한한 다양성을 제공한다.

대부분의 현대 시스템은 유연성과 메모리 효율성 때문에 온라인 증강을 선호한다.

Best Practices

증강 효과를 극대화하기 위해 다음을 고려해야 한다.

1. **시각적 검증**: 증강된 이미지 배치를 항상 직접 확인한다.
2. **타당성 우선**: 클래스 정체성을 훼손하는 변환은 피한다.
3. **작고 조합 가능한 변환 선호**: 과도한 단일 변환보다 간단한 연산을 여러 개 연결하는 것이 좋다.
4. **도메인별 조정**: 숫자 데이터에 적합한 증강이 의료 이미지나 교통 표지판에는 적합하지 않을 수 있다.

Summary

데이터 증강은 모델의 강건성을 향상시키는 데 있어 가장 강력하고 접근하기 쉬운 기법 중 하나이다. 증강은 데이터의 다양성을 모방하고, 불변성을 강화하며, 수작업으로 설계된 정규화 기법에 대한 의존도를 줄여준다.
아키텍처 변경이나 손실 함수 조정과 달리, 증강은 데이터 자체를 변형하여 모델이 실제 세계—즉, 노이즈가 많고, 변화무쌍하며, 예측 불가능한 환경—를 더 잘 마주하도록 돕는다.
잘 설계된 증강 파이프라인은 단순히 학습 데이터를 풍부하게 만드는 데 그치지 않고, 모델의 관점을 확장시킨다. 이는 네트워크가 단순히 암기하는 것이 아니라 '이해'하도록 준비시키는 과정이다.